



Machine Vision Software : MATLAB

FRA 626 Machine Vision in Smart Factory

Suriya Natsupakpong, PhD

Institute of Field Robotics (FIBO)

King Mongkut's University of Technology Thonburi (KMUTT)

Why is Machine Vision important now?

- Reducing cost of cameras and improvement in image recognition accuracy in the past decade, resulted in more accurate and cheaper machine vision systems.
- Industrial companies are aiming to achieve increased automation and efficiency with their industry 4.0 initiatives that aim to improve industrial processes with computing technology. Since the rise of deep learning, machine vision products can deliver on this promise and as a result industrial companies have been investing into these systems.
- Bloomberg estimates machine vision market size to be \$18.24 billion by 2025.
- Mckinsey expects Industry 4.0 to deliver between \$1.2 trillion and \$3.7 trillion in value potential reach in 2025, globally. It is also expected industry 4.0 to create value equivalent to efficiency improvements of 15-20%.

Machine Vision Company

- 3D Infotech
- Aquifi
- Cognex
- Datalogic
- Industrial Vision
- IVISYS
- LMI Technologies
- Microscan
- National Instruments
- Optotune
- Prophesee
- ProPhotnix
- Sensory
- Stemmer Imaging
- USS Vision
- VAIA Technologies
- ViDi Systems

Machine Vision Software

- Nation Instrument Visual Builder for Automated Instruction (AI)
- Cognex VisionPro
- Omron Visionscape
- Azure Computer Vision
- Amazon Rekognition
- Clarifai
- Google Cloud Vision API
- IBM Watson Visual Recognition
- Microsoft
 - Azure Face API
 - Emotion API
 - Computer Vision API
 - Video API
- OpenCV
- SimpleCV
- Scikit-image
- Pypylon
- CiRA CORE

MATLAB Software and Toolboxes

An interactive mathematical software environment that makes linear algebra, data analysis and high-quality graphics a breeze. A software package developed by The MathWorks Inc. Its support for Toolboxes which are collections of functions targeted at particular topics.



Data Analysis

Explore, model, and analyze data



Graphics

Visualize and explore data



Algorithm Development

Design algorithms for desktop and embedded applications



App Building

Create desktop and web apps



Using MATLAB with Other Languages

Use MATLAB with Python, C/C++, Fortran, Java, and other languages



Hardware

Connect MATLAB to hardware



Parallel Computing

Perform large-scale computations and parallelize simulations using multicore desktops, GPUs, clusters, and clouds



Web and Desktop Deployment

Share your MATLAB programs



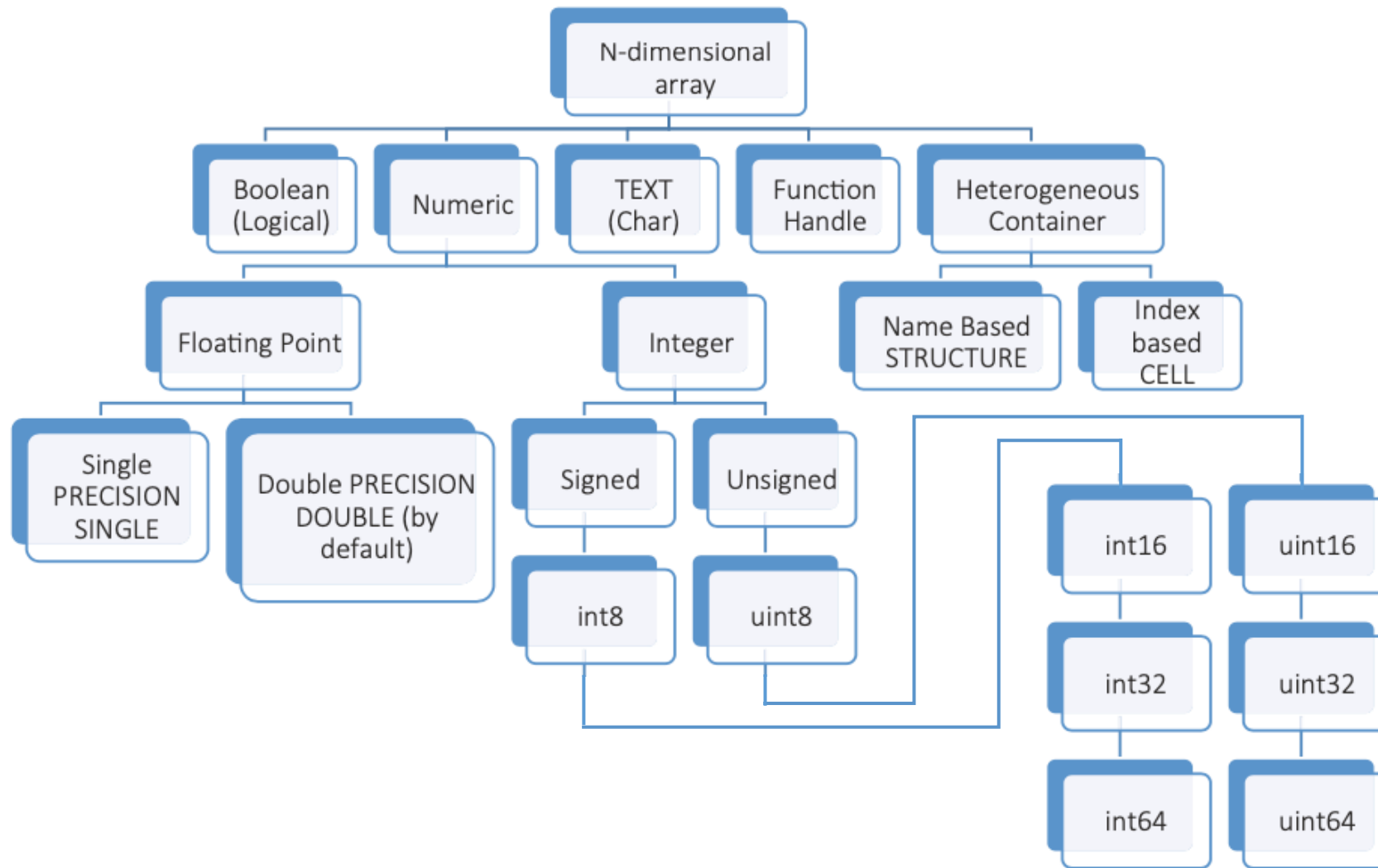
MATLAB in the Cloud

Run in cloud environments from MathWorks Cloud to public clouds including AWS and Azure

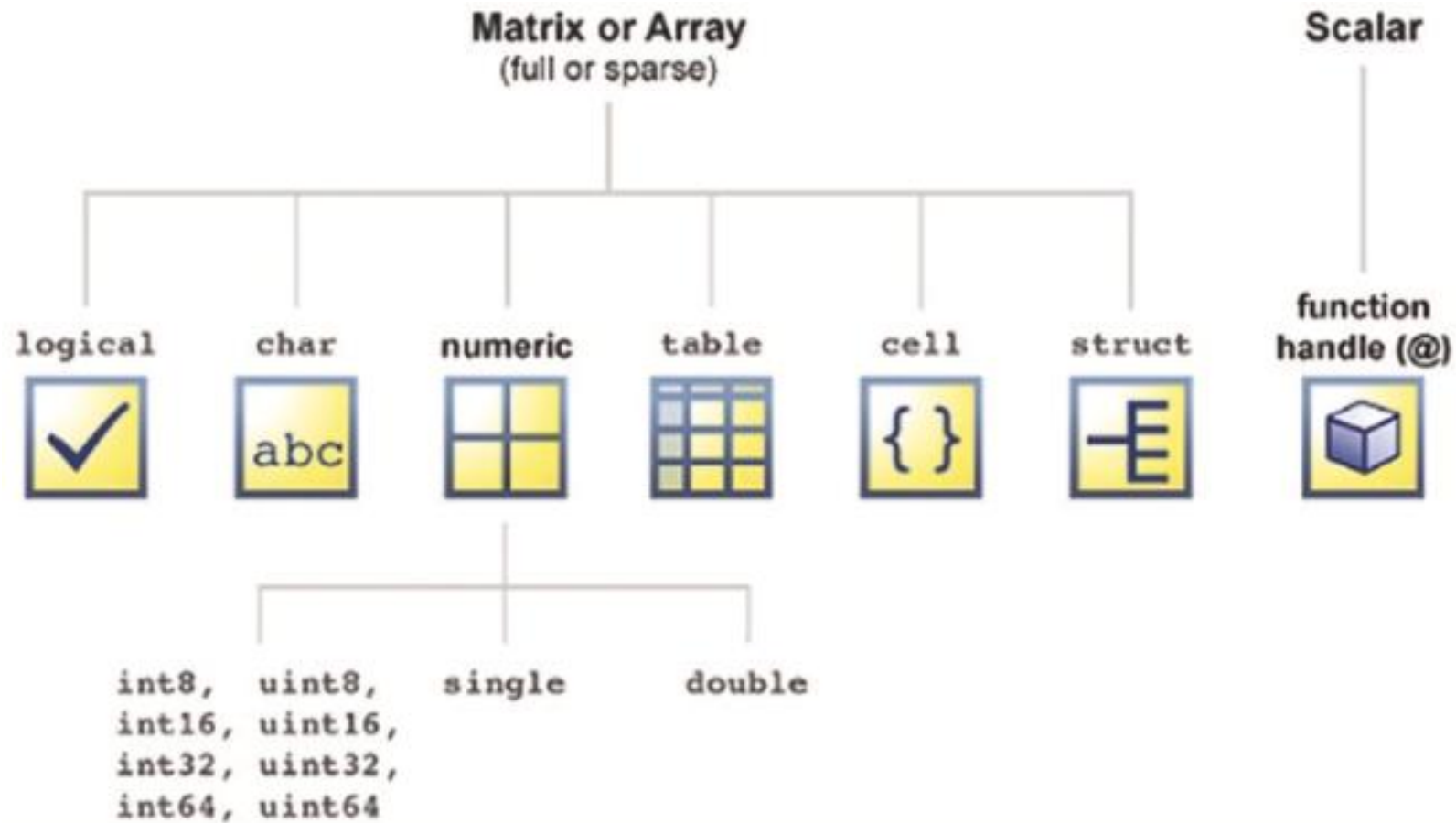
MATLAB Software and Toolboxes

- Computer Vision Toolbox™ provides algorithms, functions, and apps for designing and testing computer vision, 3D vision, and video processing systems.
- You can perform object detection and tracking, as well as feature detection, extraction, and matching.
- For 3D vision, the toolbox supports single, stereo, and fisheye camera calibration; stereo vision; 3D reconstruction; and lidar and 3D point cloud processing.
- Computer vision apps automate ground truth labeling and camera calibration workflows.
- You can train custom object detectors using deep learning and machine learning algorithms such as YOLO v2, Faster R-CNN, and ACF.
- You can accelerate your algorithms by running them on multicore processors and GPUs. Most toolbox algorithms support C/C++ code generation for integrating with existing code, desktop prototyping, and embedded vision system deployment.

Data Storage Types and Data Formats in MATLAB



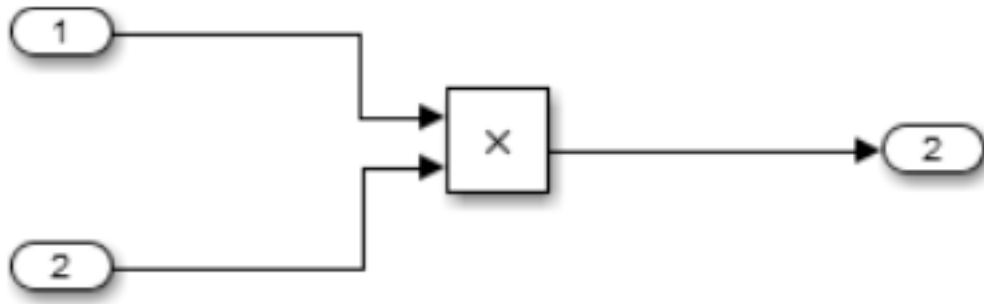
Variables and Data Sets in MATLAB



MATLAB Simulink

- A block diagram environment for multidomain simulation and Model-Based Design.
- It supports system-level design, simulation, automatic code generation, and continuous test and verification of embedded systems.
- Simulink provides a graphical editor, customizable block libraries, and solvers for modeling and simulating dynamic systems.
- It is integrated with MATLAB®, enabling you to incorporate MATLAB algorithms into models and export simulation results to MATLAB for further analysis.
- A block can represent a physical component, a small system, or a function. An input/output relationship fully characterizes a block.

Graphical Programming



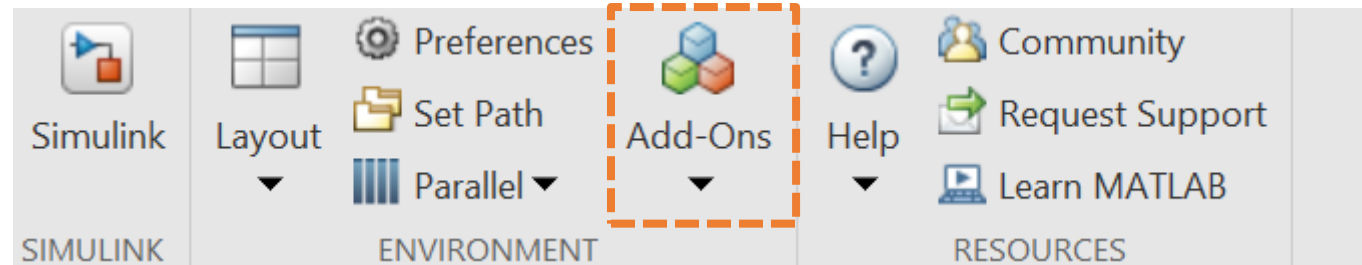
```
#include <stdio.h>
#include <stdlib.h>

int mul(int, int);

int main()
{
    int u1, u2, y1;
    printf("Enter two numbers\n");
    scanf("%d%d", &u1, &u2);
    y1 = mul(u1, u2);
    printf("Here's your result,
the operation done is a shining 'x'!\n%d\n", y1);
    return EXIT_SUCCESS;
}

int mul(int input1, int input2)
{
    return input1*input2;
}
```

MATLAB Add-Ons



Add-On Explorer

Contribute | Manage Add-Ons

vision

For You

- My Products: 110
- Recommendations: 60

Filter by Source

- MathWorks: 396
- Community: 50,263

Filter by Category

Using MATLAB

- MATLAB: 11,688

Using Simulink

- Simulink: 842
- Physical Modeling: 2,215
- Event-Based Modeling: 68
- Real-Time Simulation and Testing: 35

Explore Your Software

See all MathWorks products you can use.

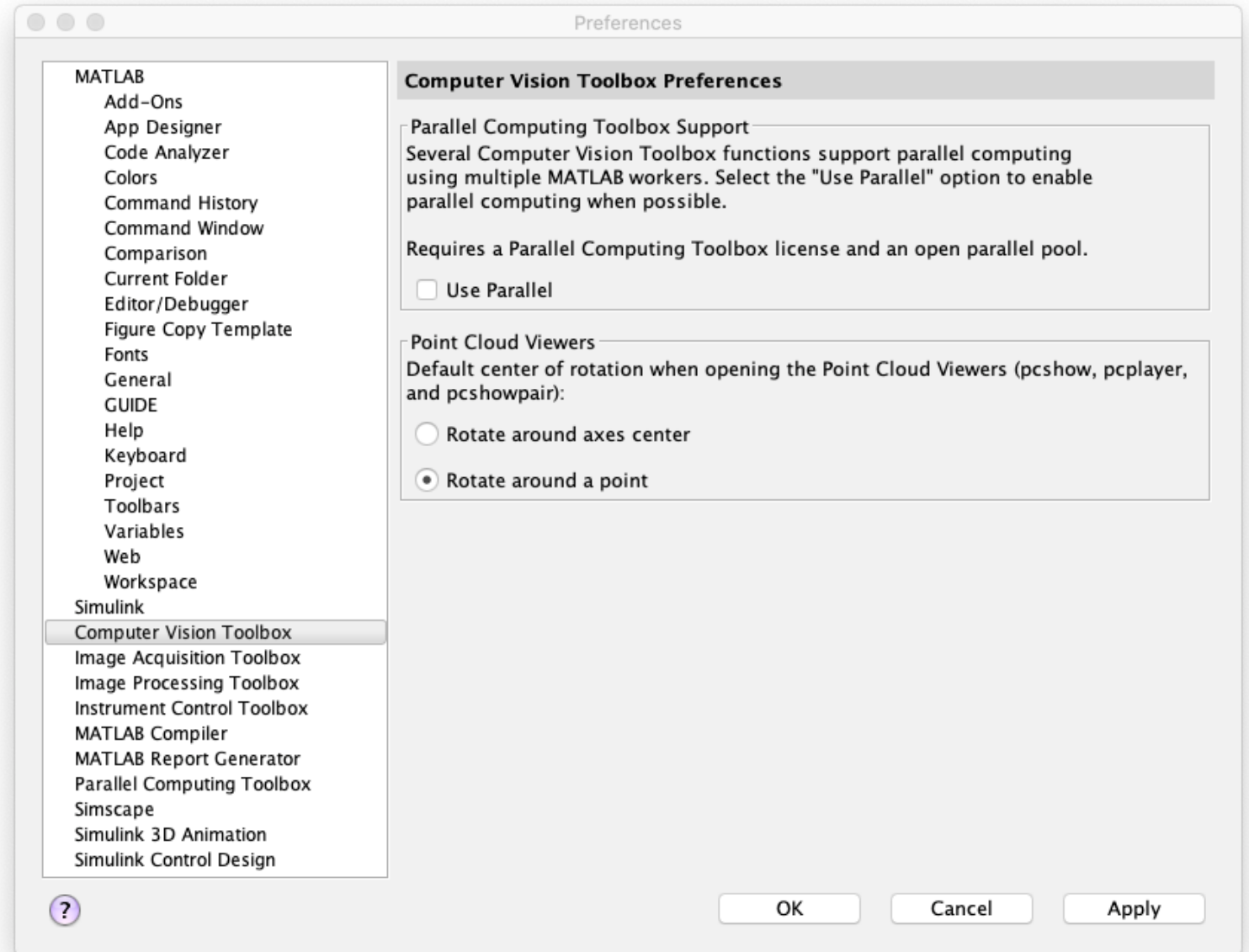
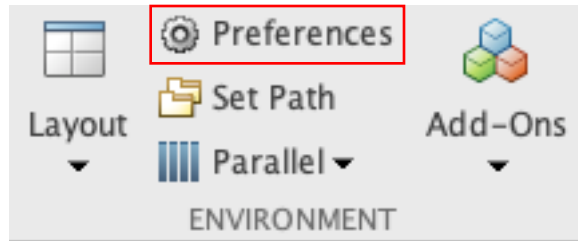
Recommended For You

- C2000 Microcontroller Blockset**
Design, simulate, and implement applicatio...
- smithchart**
A collection of files capable of drawing a...

Add-Ons

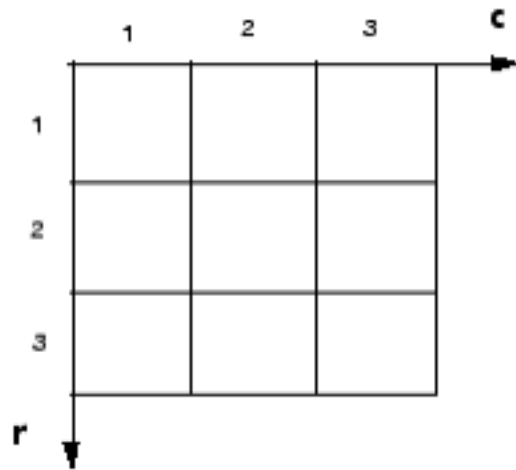
- Computer Vision Toolbox** 🚀
Design and test computer vision systems
Product
- Vision HDL Toolbox** 🚀
Design image processing, video, and com...
Product
- GPU Coder** 🚀
Generate CUDA code for NVIDIA GPUs
Product
- March 2012 demo files for "Computer Vision with MATLAB"
Demo files for "Computer Vision with MA...
5.3k Downloads ★★★★★
- Automated Visual Inspection Library for Computer Vision Toolbox 🚀
Identify anomalies or defects in images to...
836 Downloads ★★★★★

Computer Vision Toolbox Preferences

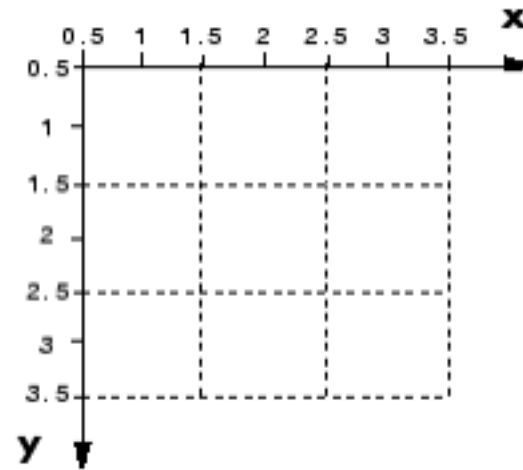


Coordinate Systems

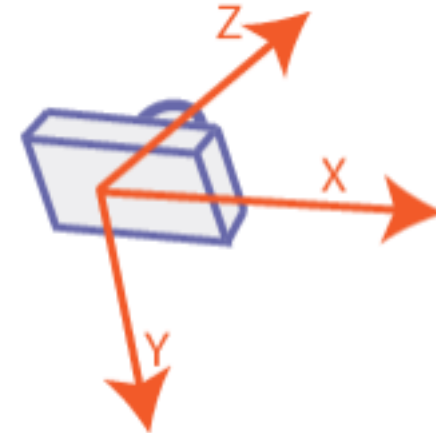
- Pixel Indices



- Spatial Coordinates



- 3-D Coordinate Systems



Computer Vision With Simulink

- Use Computer Vision Toolbox™ blocks to build models for computer vision applications. Perform feature detection, image statistics, FIR filtering, frequency and Hough transforms, morphology, contrast enhancement, and noise removal.
- Local features and their descriptors are the building blocks of many computer vision algorithms. Their applications include image registration, object detection and classification, tracking, and motion estimation.
- Motion estimation and tracking are key activities in many computer vision applications, including activity recognition, traffic monitoring, automotive safety, and surveillance.
- Analysis and enhancement techniques enable you to increase signal-to-noise ratio and accentuate features.

Object Counting

```
>> openExample('vision/videostaples')
```

Initialization

Use these next sections of code to initialize the required variables and System objects.

Create a System object to read video from avi file.

```
filename = 'staples.mp4';  
hVideoSrc = VideoReader(filename);
```

Create a blob analysis System object to count the staples and find their centroids.

```
hBlob = vision.BlobAnalysis( ...  
    'AreaOutputPort', false, ...  
    'BoundingBoxOutputPort', false, ...  
    'OutputDataType', 'single');
```

Create a System object to display the output video.

```
hVideoOut = vision.VideoPlayer('Name', 'Counted Staples');  
hVideoOut.Position(3:4) = [650 350];
```

Stream Processing Loop

Here you call the processing loop to count the staples in the input video. This loop uses the System objects you instantiated.

The loop is stopped when you reach the end of the input file, which is detected by the BinaryFileReader System object.

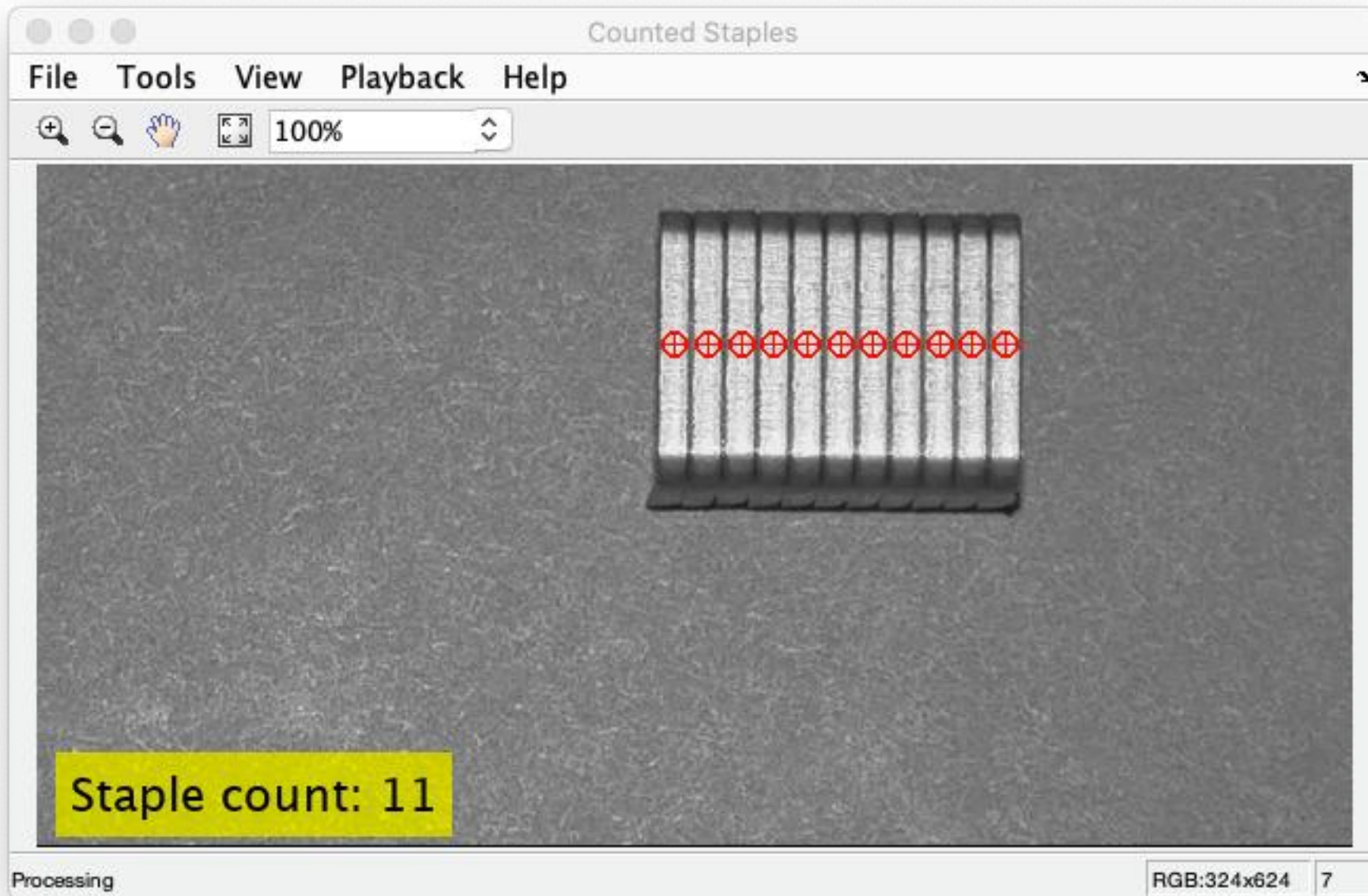
```
while hasFrame(hVideoSrc)
    I = rgb2gray(readFrame(hVideoSrc));
    Im = imtophat(I, strel('square',18));
    Im = imopen(Im, strel('rect',[15 3]));
    th = multithresh(Im); % Determine threshold using Otsu's method
    BW = Im > th;
    Centroids = step(hBlob, BW); % Blob Analysis

    StaplesCount = int32(size(Centroids,1));
    txt = sprintf('Staple count: %d', StaplesCount);
    It = insertText(I,[10 280],txt,'FontSize',22); % Display staples count

    Centroids(:, 2) = Centroids(1,2); % Align markers horizontally

    It = insertMarker(It, Centroids, 'o', 'Size', 6, 'Color', 'r');
    It = insertMarker(It, Centroids, 'o', 'Size', 5, 'Color', 'r');
    It = insertMarker(It, Centroids, '+', 'Size', 5, 'Color', 'r');

    step(hVideoOut, It);
end
```

Cell Counting

```
>> openExample('vision/videocellcounting')
```

This example shows how to use a combination of basic morphological operators and blob analysis to extract information from a video stream. In this case, the example counts the number of E. Coli bacteria in each video frame. Note that the cells are of varying brightness, which makes the task of segmentation more challenging.

Initialization

Use these next sections of code to initialize the required variables and objects.

```
VideoSize = [432 528];
```

Create a System object to read video from avi file.

```
filename = 'ecolicells.avi';  
hvfr = VideoReader(filename);
```

Create a BlobAnalysis System object to find the centroid of the segmented cells in the video.

```
hblob = vision.BlobAnalysis( ...  
    'AreaOutputPort', false, ...  
    'BoundingBoxOutputPort', false, ...  
    'OutputDataType', 'single', ...  
    'MinimumBlobArea', 7, ...  
    'MaximumBlobArea', 300, ...  
    'MaximumCount', 1500);  
  
% Acknowledgement  
ackText = ['Data set courtesy of Jonathan Young and Michael Elowitz, ' ...  
    'California Institute of Technology'];
```

Stream Processing Loop

Create a processing loop to count the number of cells in the input video. This loop uses the System objects you instantiated above.

```
frameCount = int16(1);
while hasFrame(hvfr)
    % Read input video frame
    image = rgb2gray(im2single(readFrame(hvfr)));

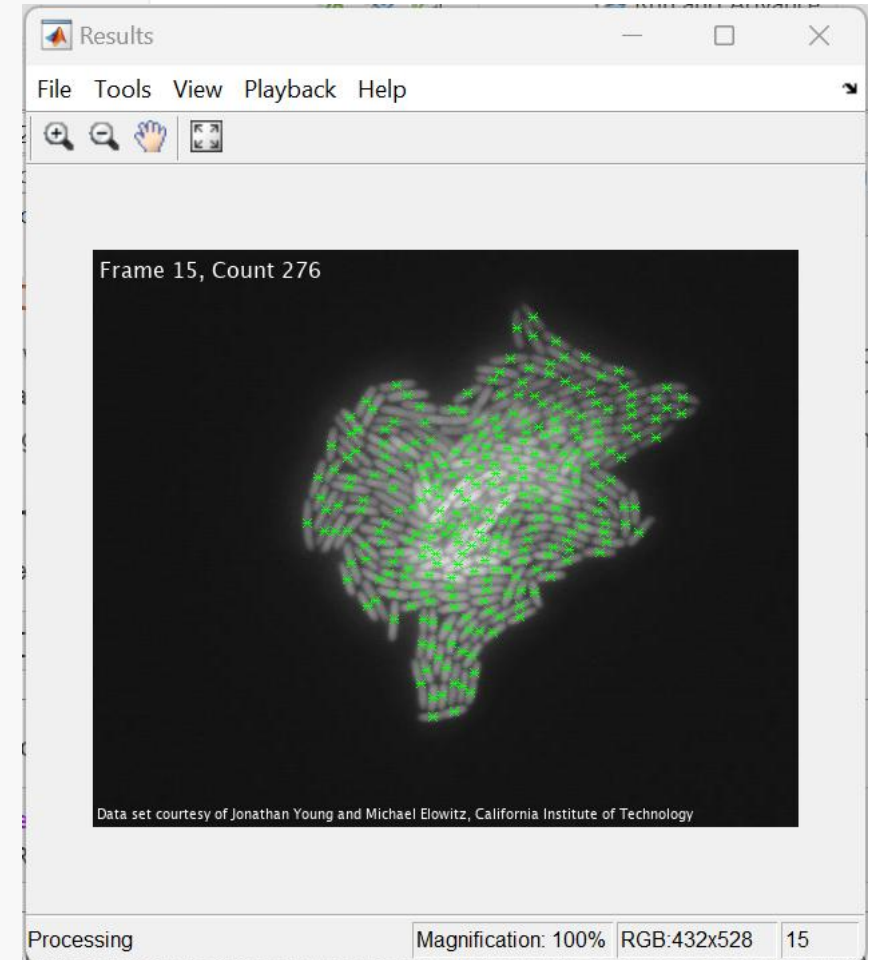
    % Apply a combination of morphological dilation and image arithmetic
    % operations to remove uneven illumination and to emphasize the
    % boundaries between the cells.
    y1 = 2*image - imdilate(image, strel('square',7));
    y1(y1<0) = 0;
    y1(y1>1) = 1;
    y2 = imdilate(y1, strel('square',7)) - y1;

    th = multithresh(y2);      % Determine threshold using Otsu's method
    y3 = (y2 <= th*0.7);      % Binarize the image.

    Centroid = step(hblob, y3); % Calculate the centroid
    numBlobs = size(Centroid,1); % and number of cells.
    % Display the number of frames and cells.
    frameBlobTxt = sprintf('Frame %d, Count %d', frameCount, numBlobs);
    image = insertText(image, [1 1], frameBlobTxt, ...
        'FontSize', 16, 'BoxOpacity', 0, 'TextColor', 'white');
    image = insertText(image, [1 size(image,1)], ackText, ...
        'FontSize', 10, 'AnchorPoint', 'LeftBottom', ...
        'BoxOpacity', 0, 'TextColor', 'white');

    % Display video
    image_out = insertMarker(image, Centroid, '*', 'Color', 'green');
    step(hVideo, image_out);

    frameCount = frameCount + 1;
end
```

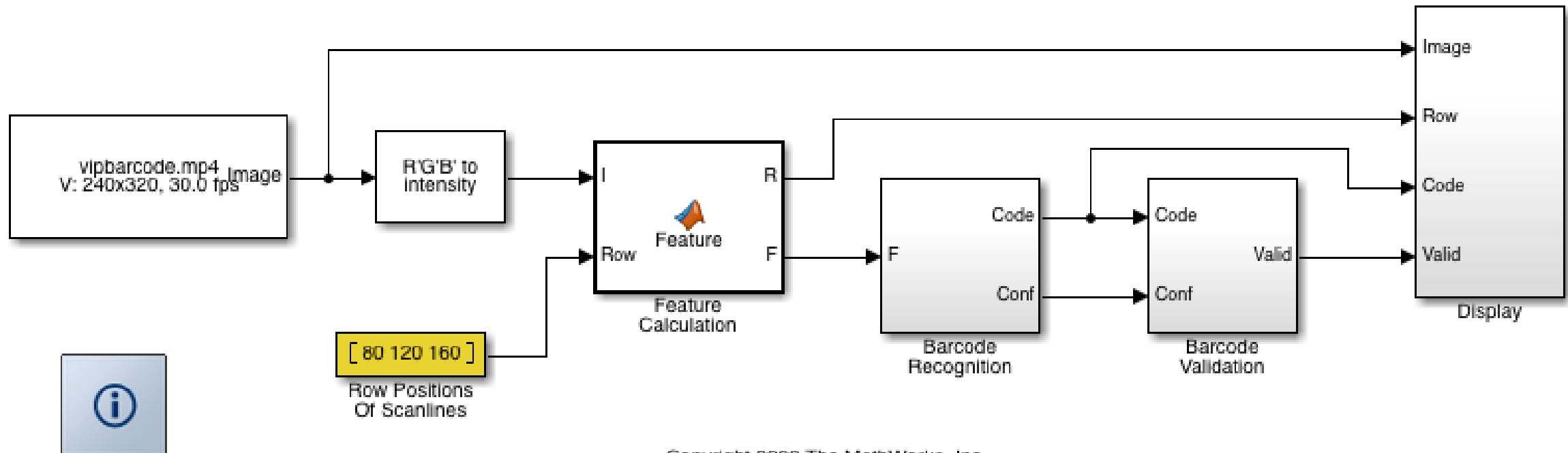


Barcode Recognition

```
>> openExample('vision/viplivebarcoderecognitionExample')
```

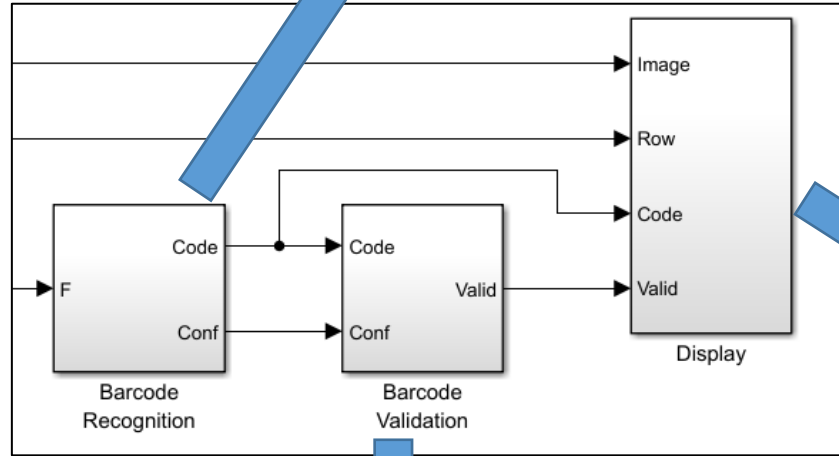
Barcode Recognition

Works with GTIN-13, EAN-13, and UPC barcodes



Copyright 2008 The MathWorks, Inc.

Barcode Recognition



Barcode Validation

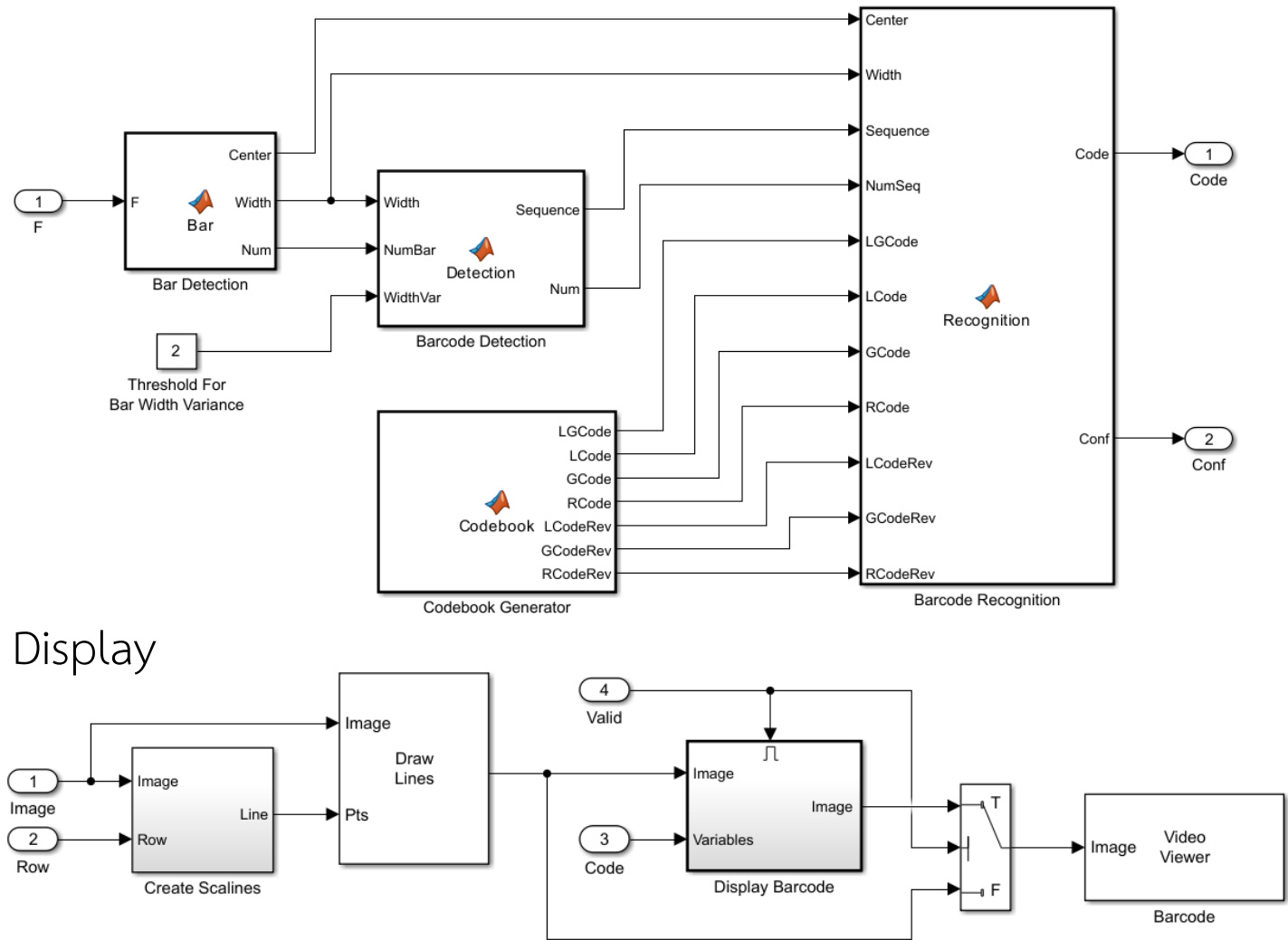
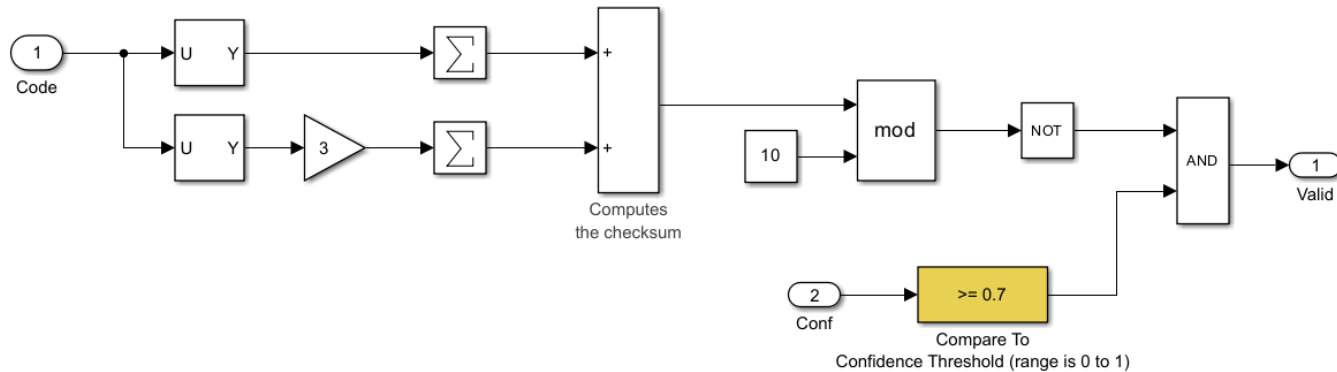


Image Acquisition Toolbox

- Open Add-On Explorer and searching for video capture devices



Image Acquisition Toolbox Support Package for OS Generic Video Interface by MathWorks Image Acquisition Toolbox Team **STAFF**

Acquire video and images from generic video capture devices.

Hardware Support



42.1K Downloads ⓘ
Updated 14 Sep 2022



Simple Video Camera Frame Grabber Toolkit Version 1.0.0.0 by Kenneth Perrine

A toolkit for **capturing** still frames from Firewire or USB **video** cameras

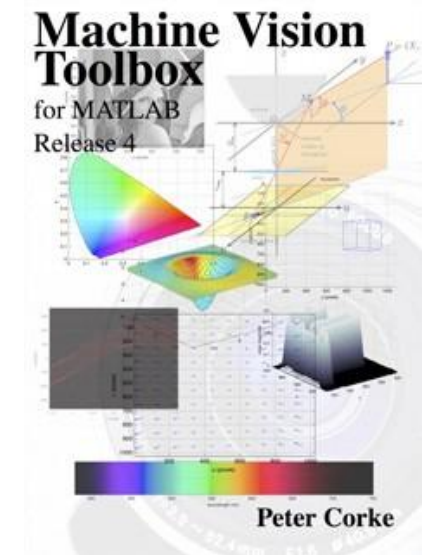
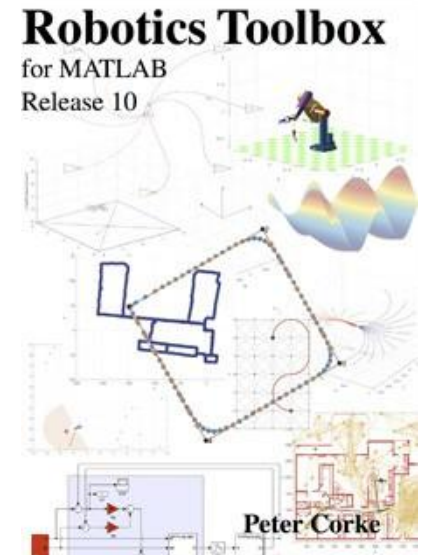
multiple **video devices**! It's easy! Here is an example MATLAB script for **capturing** a **video** frame from the default **capture device**:
`fgrabm_start % Initialize the framework and start capture pause(2) %`



3.1K Downloads ⓘ
Updated 10 Mar 2012

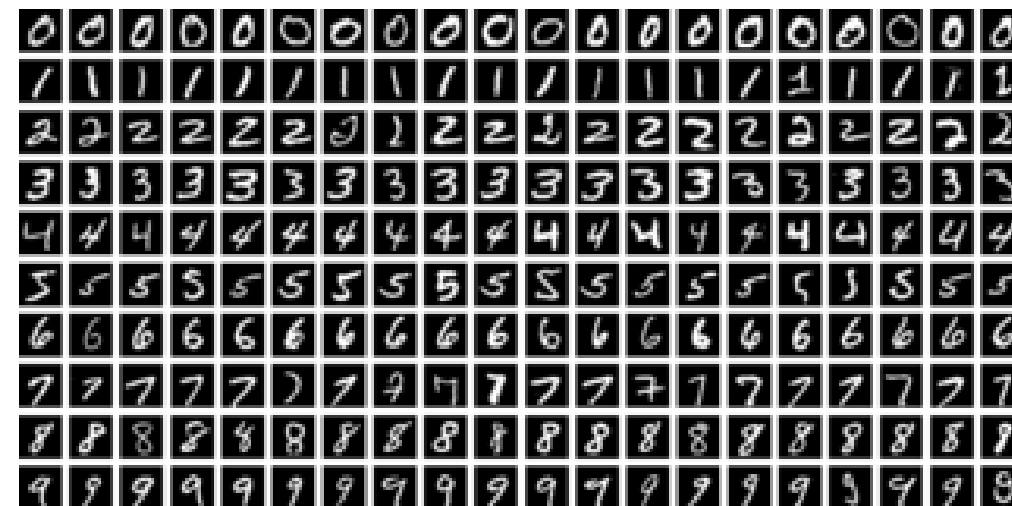
MATLAB Toolboxes for Robotics and Machine Vision

- Peter Corke has been developed two toolboxes: <https://petercorke.com/>
 - The Robotics Toolbox (RTB) provides a diverse range of functions for simulating mobile and arm-type robots.
 - The Machine Vision Toolbox (MVTB) provides a rich collection of functions for camera modeling, image processing, image feature extraction, multi-view geometry and vision-based control.
- The MVTB also contains functions for image acquisition and display; filtering; blob, point and line feature extraction; mathematical morphology; image warping; stereo vision; homography and fundamental matrix estimation; robust estimation; bundle adjustment; visual Jacobians; geometric camera models; camera calibration and color space operations. For modest image sizes on a modern computer the processing rate can be sufficiently “real-time” to allow for closed-loop control.

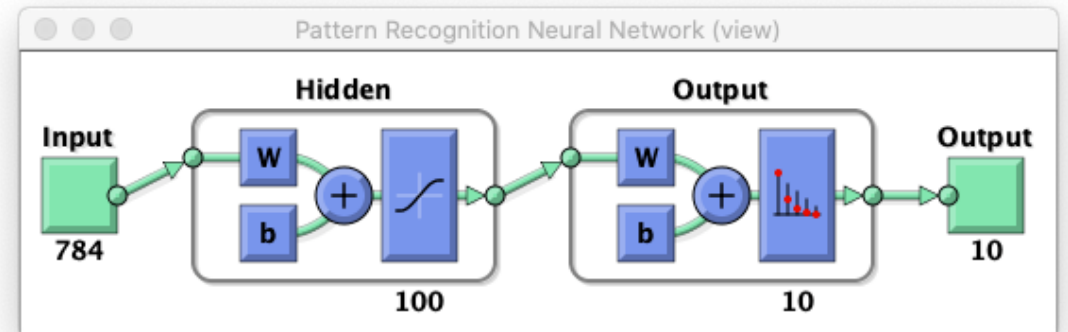
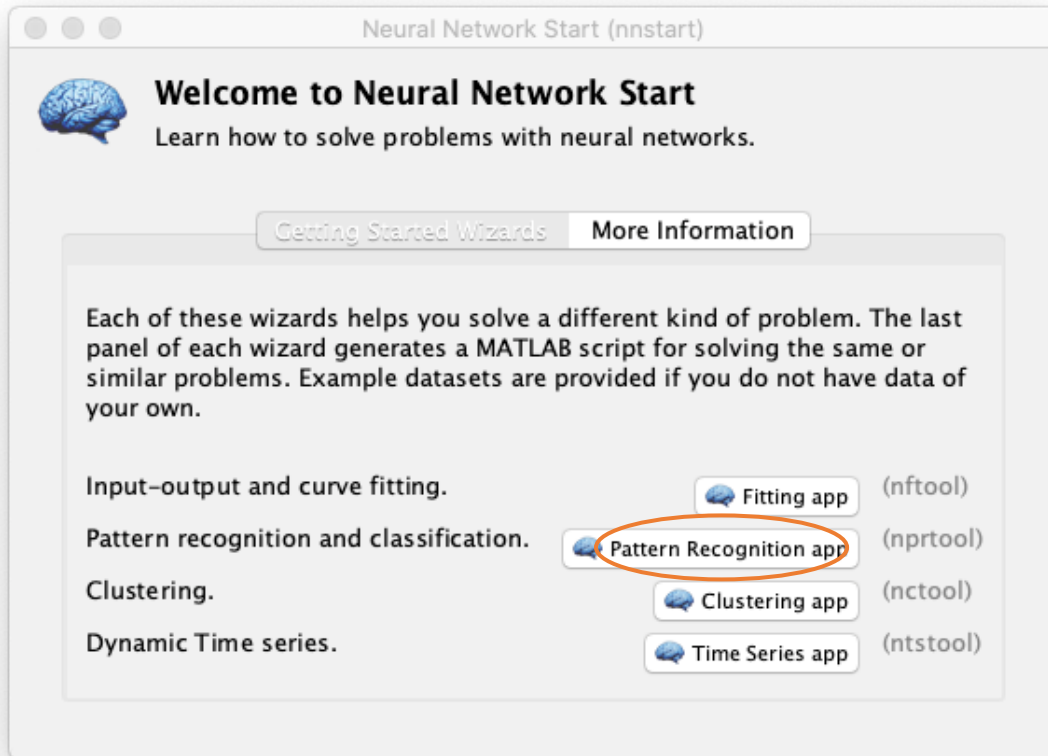


THE MNIST DATABASE of Handwritten Digits

- Yann Lecunn ผู้บุกเบิกวิจัยทางด้าน Computer Vision
- Dataset: <https://www.kaggle.com/c/digit-recognizer/data>
- The digits have been size-normalized and centered in a fixed-size 28x28 image. A training set of 60,000 examples, and a test set of 10,000 examples.
- Follow the instruction in <https://blogs.mathworks.com/loren/2015/08/04/artificial-neural-networks-for-beginners/>



Neural Network Start : nnstart





Welcome to the Neural Network Pattern Recognition app.

Solve a pattern-recognition problem with a two-layer feed-forward network.

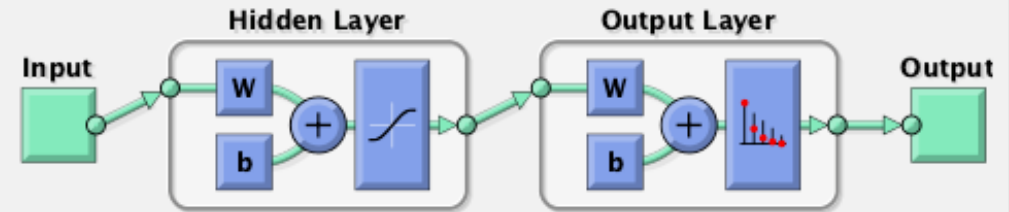
Introduction

In pattern recognition problems, you want a neural network to classify inputs into a set of target categories.

For example, recognize the vineyard that a particular bottle of wine came from, based on chemical analysis ([wine_dataset](#)); or classify a tumor as benign or malignant, based on uniformity of cell size, clump thickness, mitosis ([cancer_dataset](#)).

The Neural Pattern Recognition app will help you select data, create and train a network, and evaluate its performance using cross-entropy and confusion matrices.

Neural Network



A two-layer feed-forward network, with sigmoid hidden and softmax output neurons ([patternnet](#)), can classify vectors arbitrarily well, given enough neurons in its hidden layer.

The network will be trained with scaled conjugate gradient backpropagation ([trainscg](#)).



To continue, click [Next].

Neural Network Start

Welcome

Back

Next

Cancel



Select Data

What inputs and targets define your pattern recognition problem?

Get Data from Workspace

Input data to present to the network.

Inputs:

Xtrain



...

Target data defining desired network output.

Targets:

Ytrain



...

Samples are:



Matrix columns



Matrix rows

Want to try out this tool with an example data set?

Load Example Data Set

Summary

Inputs 'Xtrain' is a 784x28000 matrix, representing static data: 28000 samples of 784 elements.

Targets 'Ytrain' is a 10x28000 matrix, representing static data: 28000 samples of 10 elements.



To continue, click [Next].

Neural Network Start

Welcome

Back

Next

Cancel



Validation and Test Data

Set aside some samples for validation and testing.

Select Percentages

 Randomly divide up the 28000 samples:

 Training: 70% 19600 samples

 Validation: 15% 4200 samples

 Testing: 15% 4200 samples

Restore Defaults

Explanation

 Three Kinds of Samples:

 Training:

These are presented to the network during training, and the network is adjusted according to its error.

 Validation:

These are used to measure network generalization, and to halt training when generalization stops improving.

 Testing:

These have no effect on training and so provide an independent measure of network performance during and after training.



Change percentages if desired, then click [Next] to continue.

 Neural Network Start

 Welcome

 Back

 Next

 Cancel



Network Architecture

Set the number of neurons in the pattern recognition network's hidden layer.

Hidden Layer

Define a pattern recognition neural network. (patternnet)

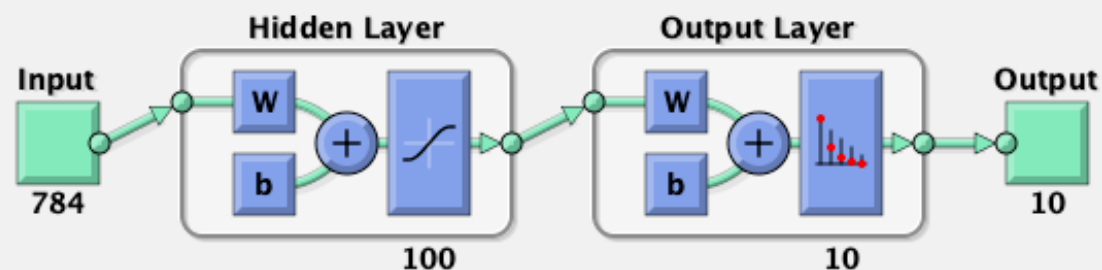
Number of Hidden Neurons:

Restore Defaults

Recommendation

Return to this panel and change the number of neurons if the network does not perform well after training.

Neural Network



Change settings if desired, then click [Next] to continue.

Neural Network Start

Welcome

Back

Next

Cancel

Neural Pattern Recognition

Train Network

Train the network to classify the inputs according to the targets.

Train Network

Train using scaled conjugate gradient backpropagation. (trainscg)

 Train

Training automatically stops when generalization stops improving, as indicated by an increase in the cross-entropy error of the validation samples.

Notes

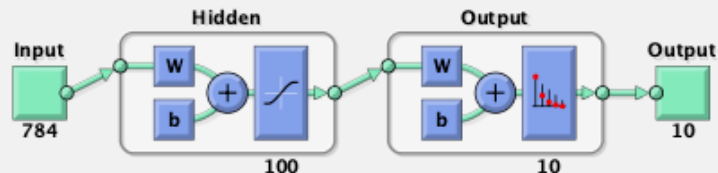
 Training multiple times will generate different results due to different initial conditions and sampling.

Training network.

Neural Network Start Welcome

Neural Network Training (nntraintool)

Neural Network



Algorithms

Data Division: Random (dividerand)
Training: Scaled Conjugate Gradient (trainscg)
Performance: Cross-Entropy (crossentropy)
Calculations: MEX

Progress

Epoch:	0		95 iterations	1000
Time:			0:01:42	
Performance:	0.664		0.00219	0.00
Gradient:	3.64		0.00515	1.00e-06
Validation Checks:	0		6	6

Plots

Performance (plotperform)
Training State (plottrainstate)
Error Histogram (ploterrhist)
Confusion (plotconfusion)
Receiver Operating Characteristic (plotroc)

Plot Interval: 1 epochs

Validation stop.

Stop Training Cancel



Train Network

Train the network to classify the inputs according to the targets.

Train Network

Train using scaled conjugate gradient backpropagation. (trainscg)



Training automatically stops when generalization stops improving, as indicated by an increase in the cross-entropy error of the validation samples.

Notes



Training multiple times will generate different results due to different initial conditions and sampling.



Minimizing Cross-Entropy results in good classification. Lower values are better. Zero means no error.



Percent Error indicates the fraction of samples which are misclassified. A value of 0 means no misclassifications, 100 indicates maximum misclassifications.

Results

	 Samples	 CE	 %E
 Training:	19600	3.60097e-0	4.13265e-1
 Validation:	4200	10.24196e-0	3.78571e-0
 Testing:	4200	10.24316e-0	4.40476e-0

Plot Confusion

Plot ROC



Open a plot, retrain, or click [Next] to continue.

 Neural Network Start

 Welcome

 Back

 Next

 Cancel

Output Class



1 2 3 4 5 6 7 8 9 10

Output Class



1 2 3 4 5 6 7 8 9 10

Output Class



1 2 3 4 5 6 7 8 9 10

Output Class



1 2 3 4 5 6 7 8 9 10



Evaluate Network

Optionally test network on more data, then decide if network performance is good enough.

Iterate for improved performance

Try training again if a first try did not generate good results or you require marginal improvement.

 Train Again

Increase network size if retraining did not help.

 Adjust Network Size

Not working? You may need to use a larger data set.

 Import Larger Data Set

Optionally perform additional tests

 Inputs: (none)  ...

 Targets: (none)  ...

Samples are: ☒  Matrix columns ☐  Matrix rows

No inputs selected.

No targets selected.

 Test Network

 CE

 %E

Plot Confusion

Plot ROC



Select inputs and targets, click an improvement button, or click [Next].

 Neural Network Start

 Welcome

 Back

 Next

 Cancel



Deploy Solution

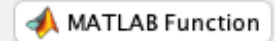
Generate deployable versions of your trained neural network.

Application Deployment

Prepare neural network for deployment with MATLAB Compiler and Builder tools.

Generate a MATLAB function with matrix and cell array argument support:

(genFunction)



Code Generation

Prepare neural network for deployment with MATLAB Coder tools.

Generate a MATLAB function with matrix-only arguments (no cell array support):

(genFunction)



Simulink Deployment

Simulate neural network in Simulink or deploy with Simulink Coder tools.

Generate a Simulink diagram:

(gensim)



Graphics

Generate a graphical diagram of the neural network:

(network/view)



Deploy a neural network or click [Next].



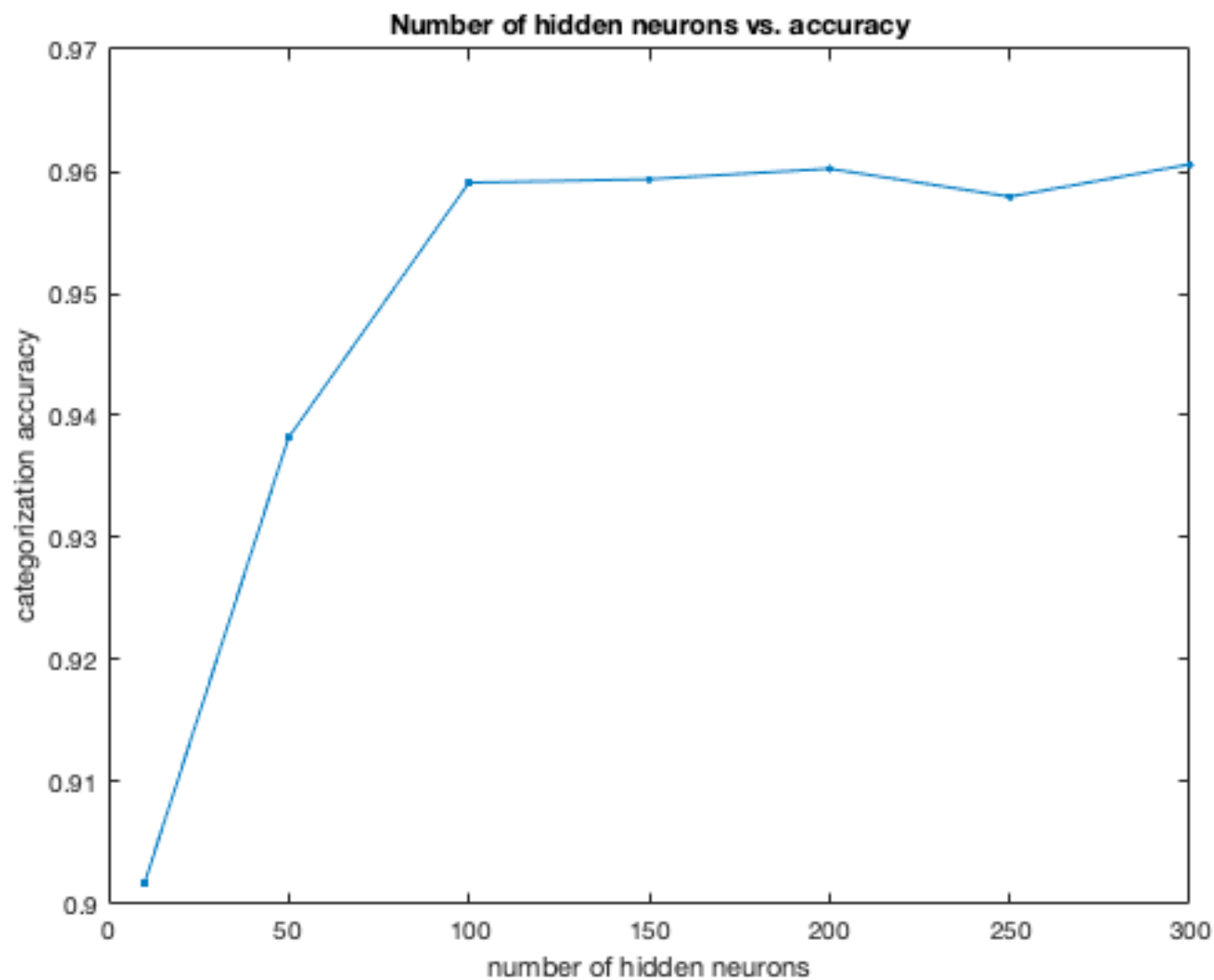
[illegible]

How many the number of hidden neurons should be?

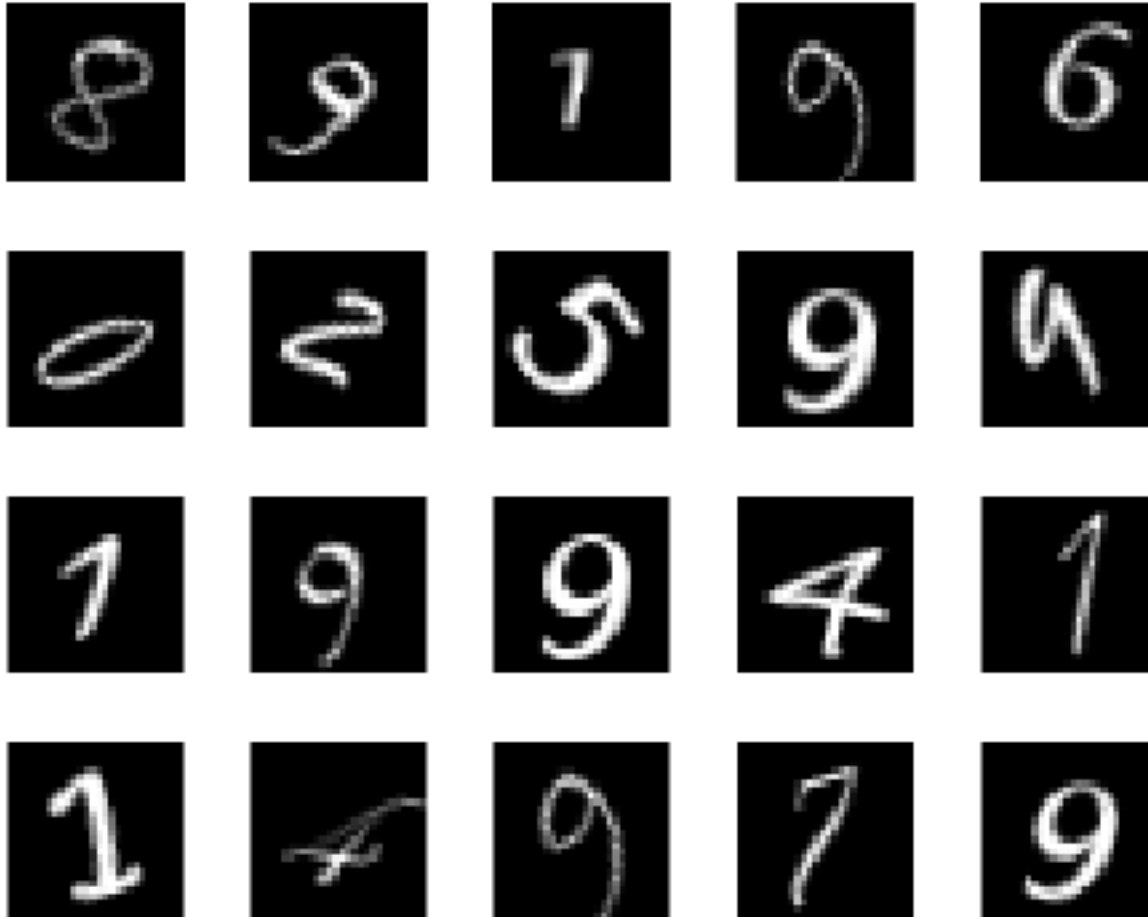
```
sweep = [10,50:50:300];           % parameter values to test
scores = zeros(length(sweep), 1); % pre-allocation
models = cell(length(sweep), 1);  % pre-allocation
x = Xtrain;                       % inputs
t = Ytrain;                       % targets
trainFcn = 'trainscg';            % scaled conjugate gradient
for i = 1:length(sweep)
    hiddenLayerSize = sweep(i);   % number of hidden layer neurons
    net = patternnet(hiddenLayerSize); % pattern recognition network
    net.divideParam.trainRatio = 70/100; % 70% of data for training
    net.divideParam.valRatio = 15/100; % 15% of data for validation
    net.divideParam.testRatio = 15/100; % 15% of data for testing
    net = train(net, x, t);       % train the network
    models{i} = net;              % store the trained network
    p = net(Xtest);               % predictions
    [~, p] = max(p);              % predicted labels
    scores(i) = sum(Ytest == p) / ... % categorization accuracy
                length(Ytest);
end
```

Result

```
figure
plot(sweep, scores, '.-')
xlabel('number of hidden neurons')
ylabel('categorization accuracy')
title('Number of hidden neurons vs. accuracy')
```



Simple Deep Learning Network for Classification

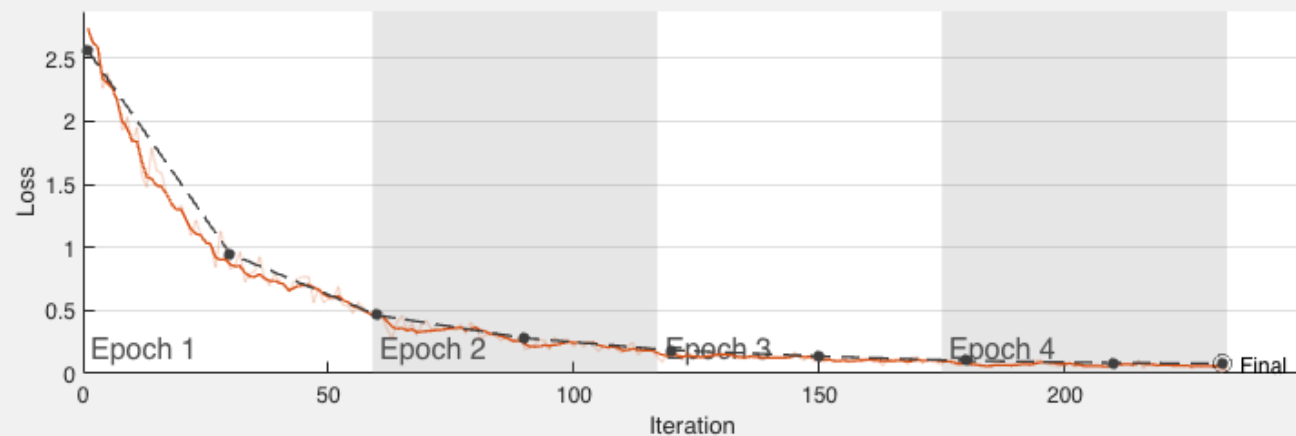
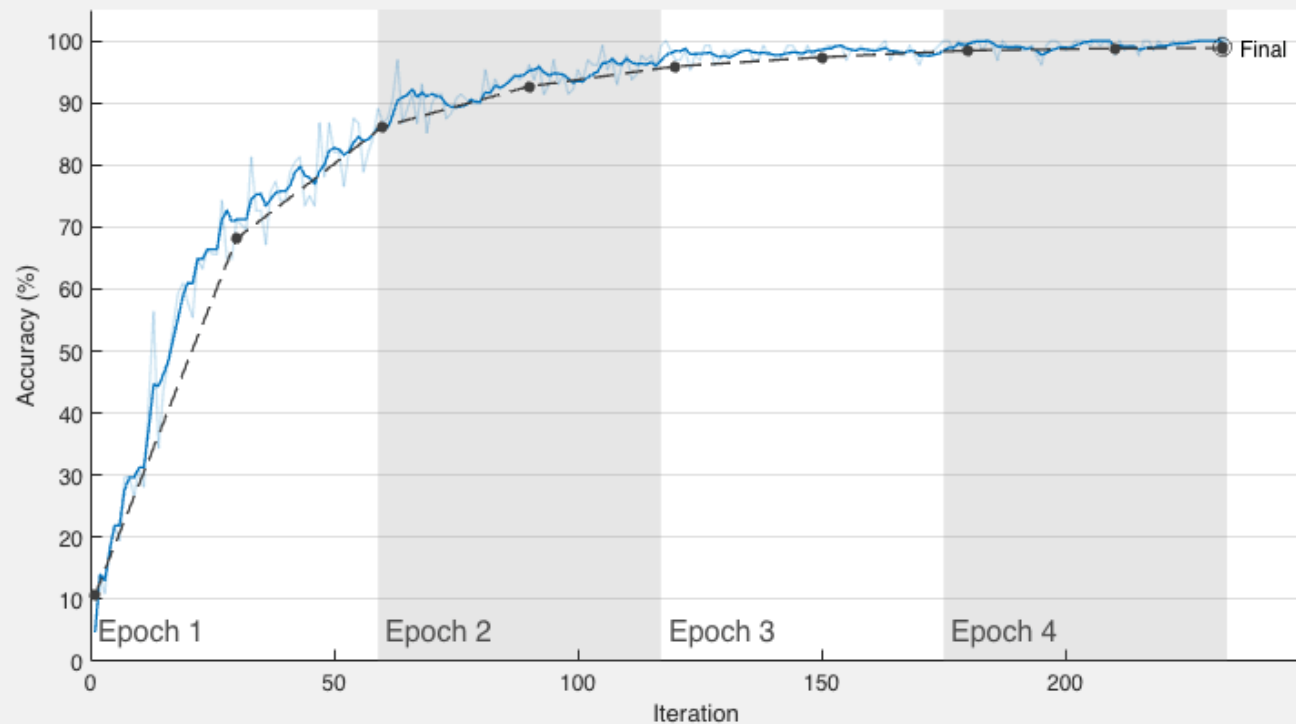


Define Network Architecture

Define the convolutional neural network architecture.

```
layers = [  
    imageInputLayer([28 28 1])  
  
    convolution2dLayer(3,8,'Padding','same')  
    batchNormalizationLayer  
    reluLayer  
  
    maxPooling2dLayer(2,'Stride',2)  
  
    convolution2dLayer(3,16,'Padding','same')  
    batchNormalizationLayer  
    reluLayer  
  
    maxPooling2dLayer(2,'Stride',2)  
  
    convolution2dLayer(3,32,'Padding','same')  
    batchNormalizationLayer  
    reluLayer  
  
    fullyConnectedLayer(10)  
    softmaxLayer  
    classificationLayer];
```

Training Progress (10-Mar-2021 21:34:55)



Results

Validation accuracy: 98.96%
Training finished: Reached final iteration

Training Time

Start time: 10-Mar-2021 21:34:55
Elapsed time: 1 min 46 sec

Training Cycle

Epoch: 4 of 4
Iteration: 232 of 232
Iterations per epoch: 58
Maximum iterations: 232

Validation

Frequency: 30 iterations

Other Information

Hardware resource: Single CPU
Learning rate schedule: Constant
Learning rate: 0.01

[i Learn more](#)

Accuracy

— Training (smoothed)
— Training
— Validation

Loss

— Training (smoothed)
— Training
— Validation

Read .CSV Data

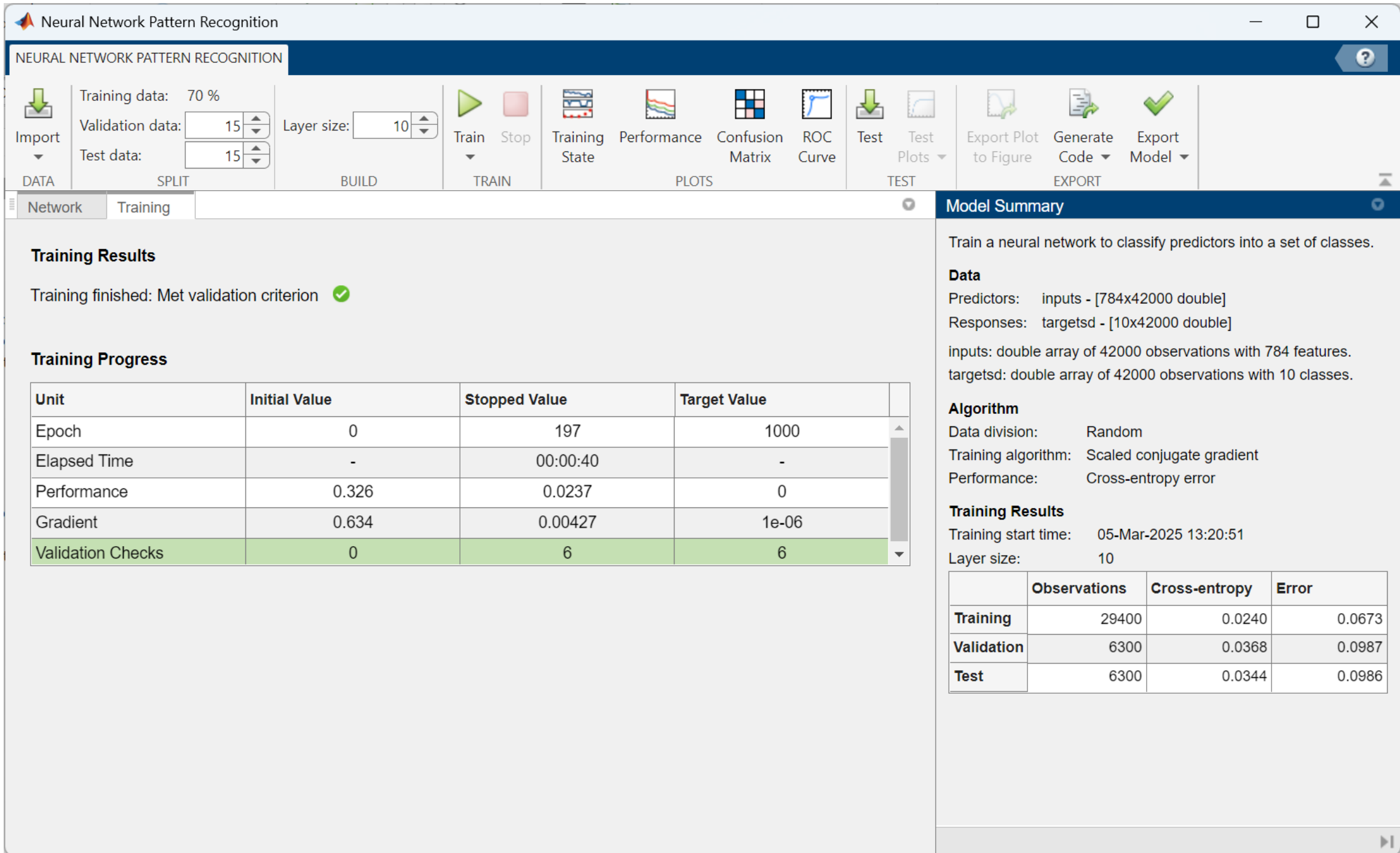
```
tr = csvread('train.csv', 1, 0); % read train.csv

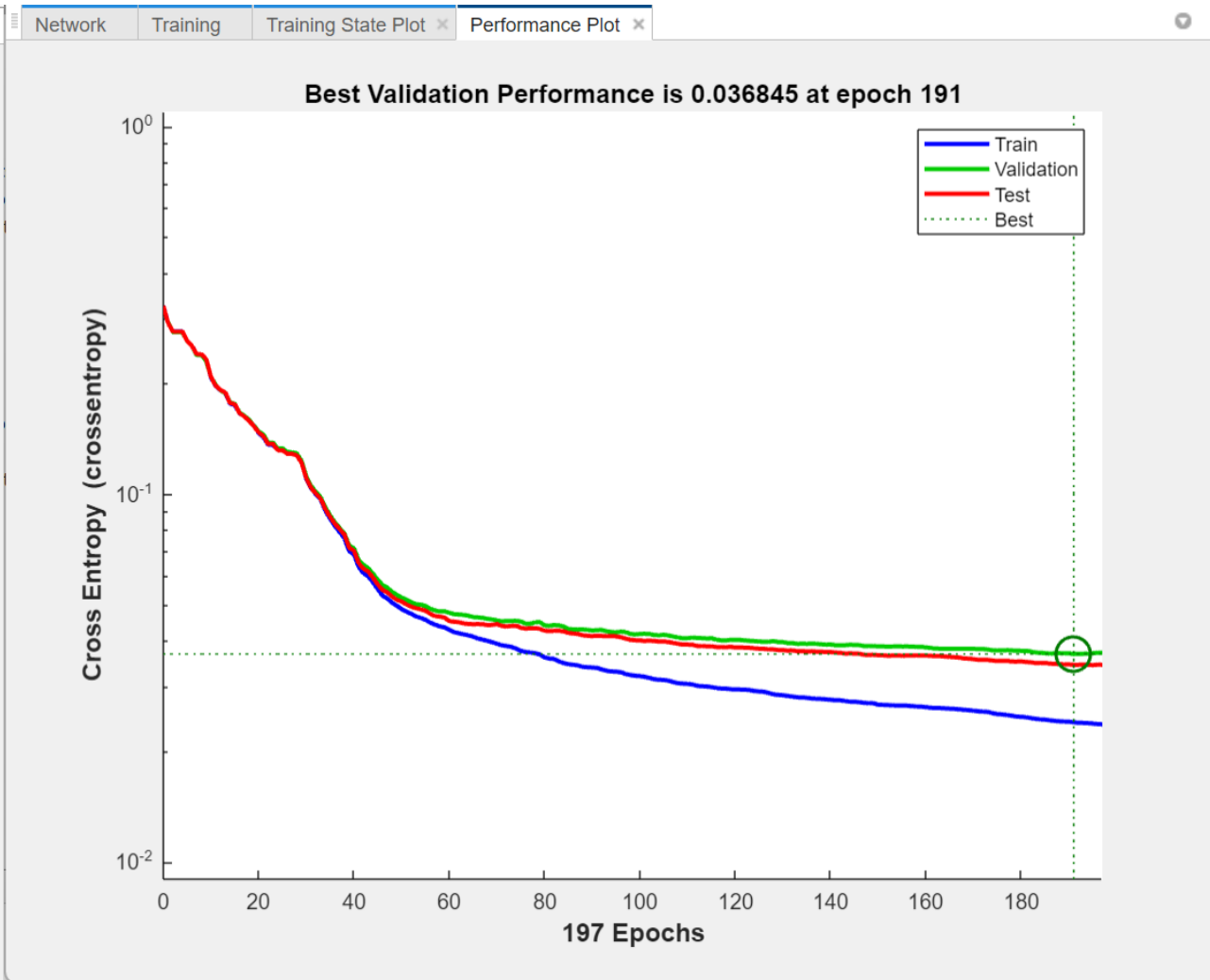
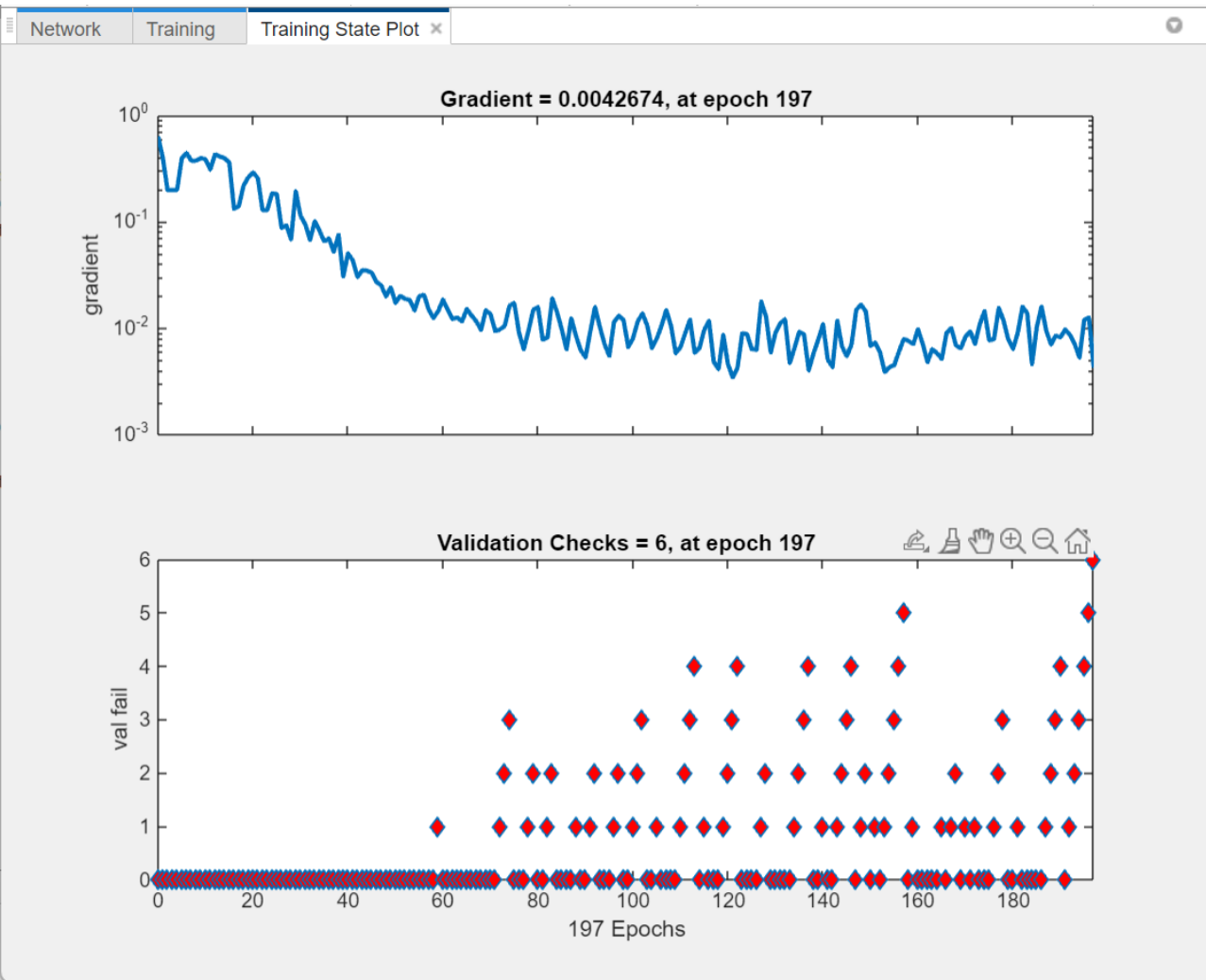
%%
figure % plot images
colormap(gray) % set to grayscale
for i = 1:25 % preview first 25 samples
    subplot(5,5,i) % plot them in 5 x 5 grid
    digit = reshape(tr(i, 2:end), [28,28]); % row = 28 x 28 image
    imagesc(digit) % show the image
    title(num2str(tr(i, 1))) % show the label
End

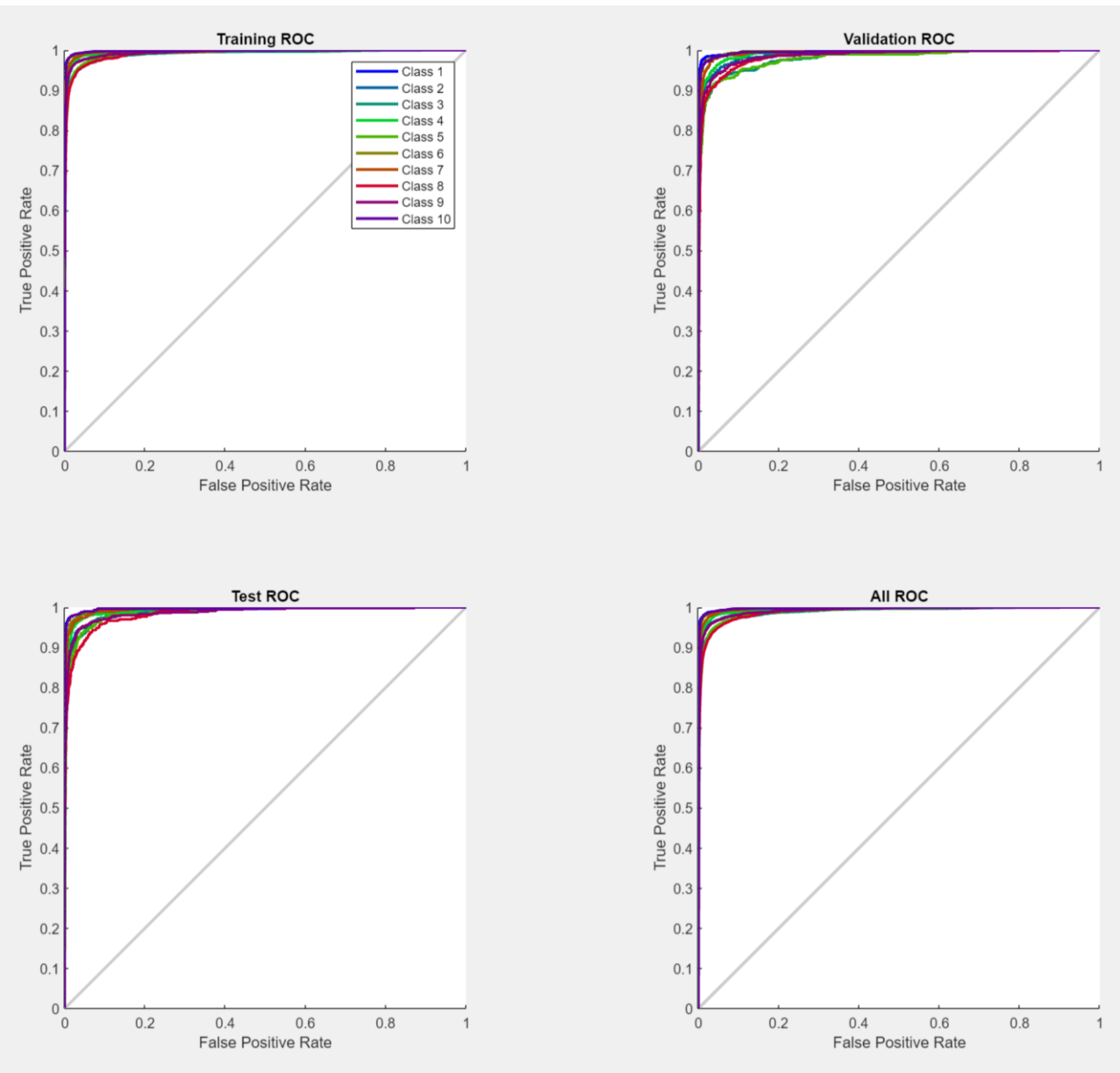
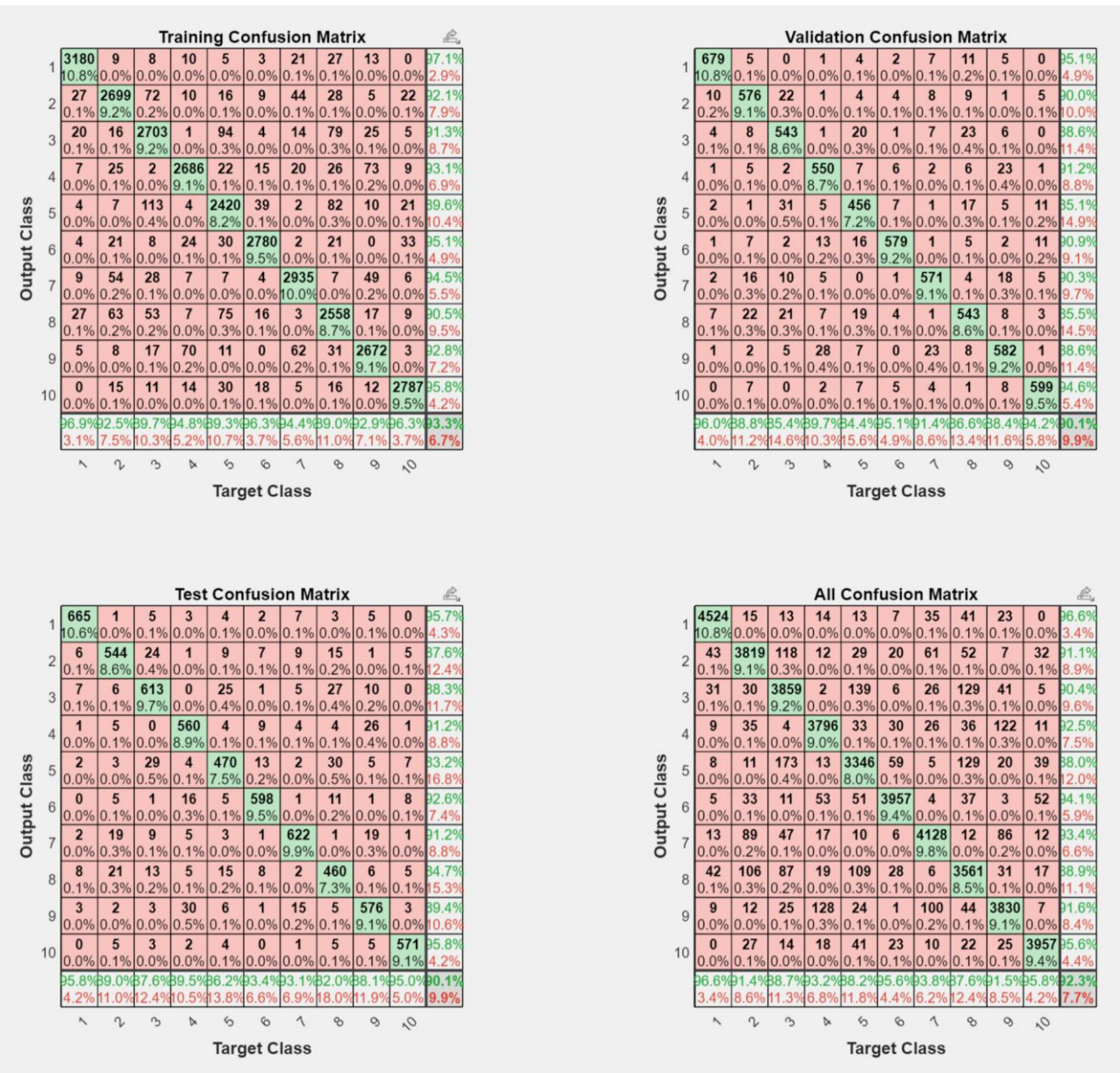
%%
n = size(tr, 1); % number of samples in the dataset
targets = tr(:,1); % 1st column is |label|
targets(targets == 0) = 10; % use '10' to present '0'
targetsd = dummyvar(targets); % convert label into a dummy variable
inputs = tr(:,2:end); % the rest of columns are predictors

inputs = inputs'; % transpose input
targets = targets'; % transpose target
targetsd = targetsd'; % transpose dummy variable

%%
nnstart
```







Test ROC

All ROC

Solving a Sudoku Puzzle Using a Webcam

- Read in a Sudoku puzzle using a USB webcam, extract data from it using image processing, and solve it using a simple numerical algorithm.
- <https://www.mathworks.com/videos/solving-a-sudoku-puzzle-using-a-webcam-68773.html>

Case Study #5 : ML with MATLAB

- ให้ลองเล่น Neural Network และ Deep learning ใน MATLAB โดยศึกษาวิธีการใช้งานได้ที่
- <https://blogs.mathworks.com/loren/2015/08/04/artificial-neural-networks-for-beginners/>
- <https://explore.mathworks.com/machine-learning-vs-deep-learning>
- โดยเลือกใช้ตัวอย่างข้อมูลแบบเดียวกับที่ใช้ใน Case Study #4: ML.NET
- ส่งผลการใช้งานได้ที่ LEB2